



AI61002: Deep Learning Foundations and Applications

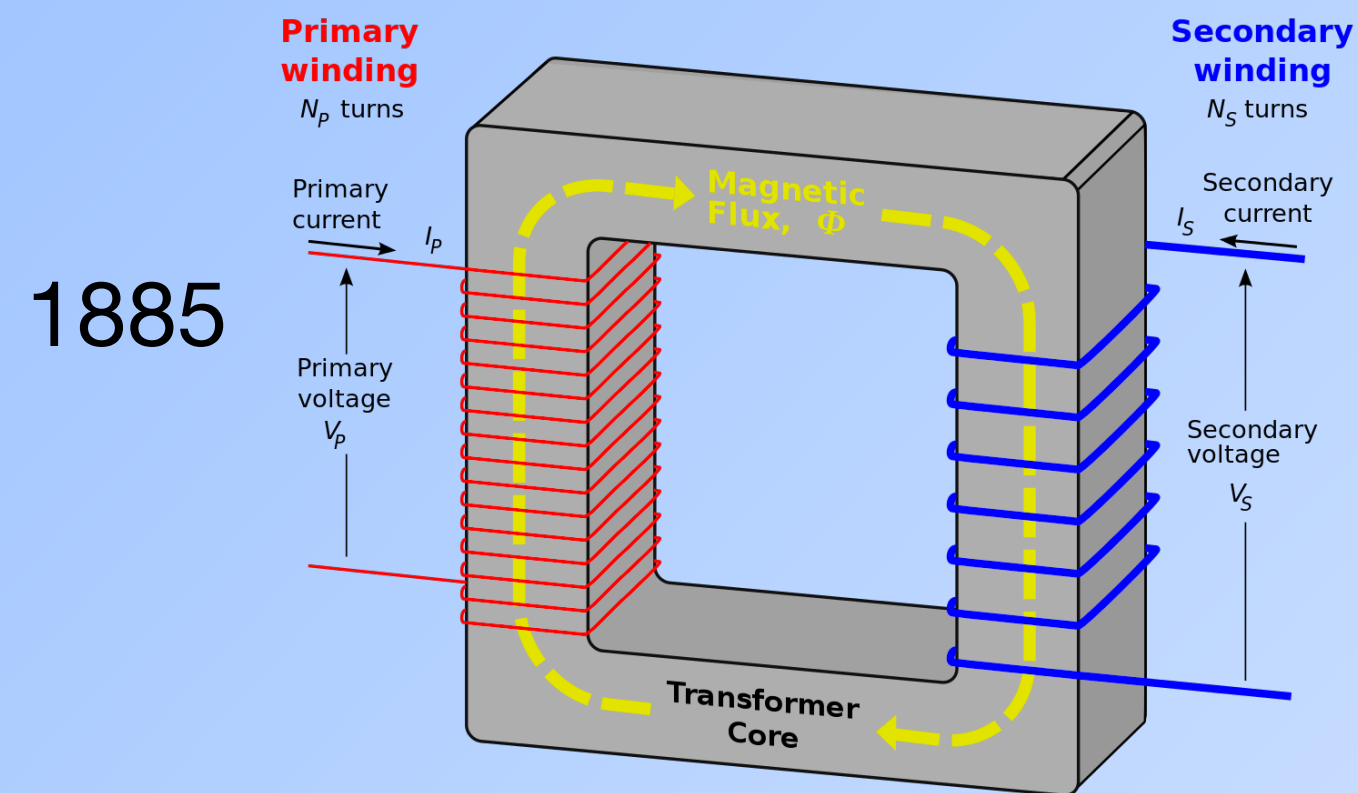
Module 7: Transformers

Dr. Somdyuti Paul

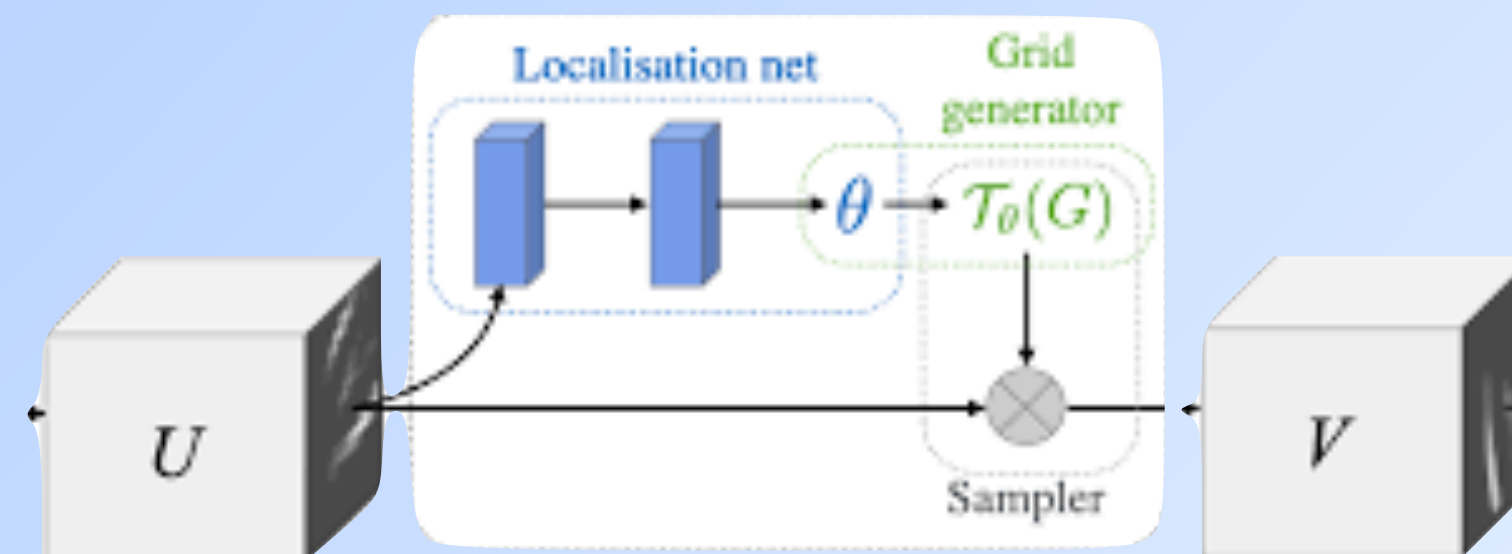
What Are Transformers?



1984

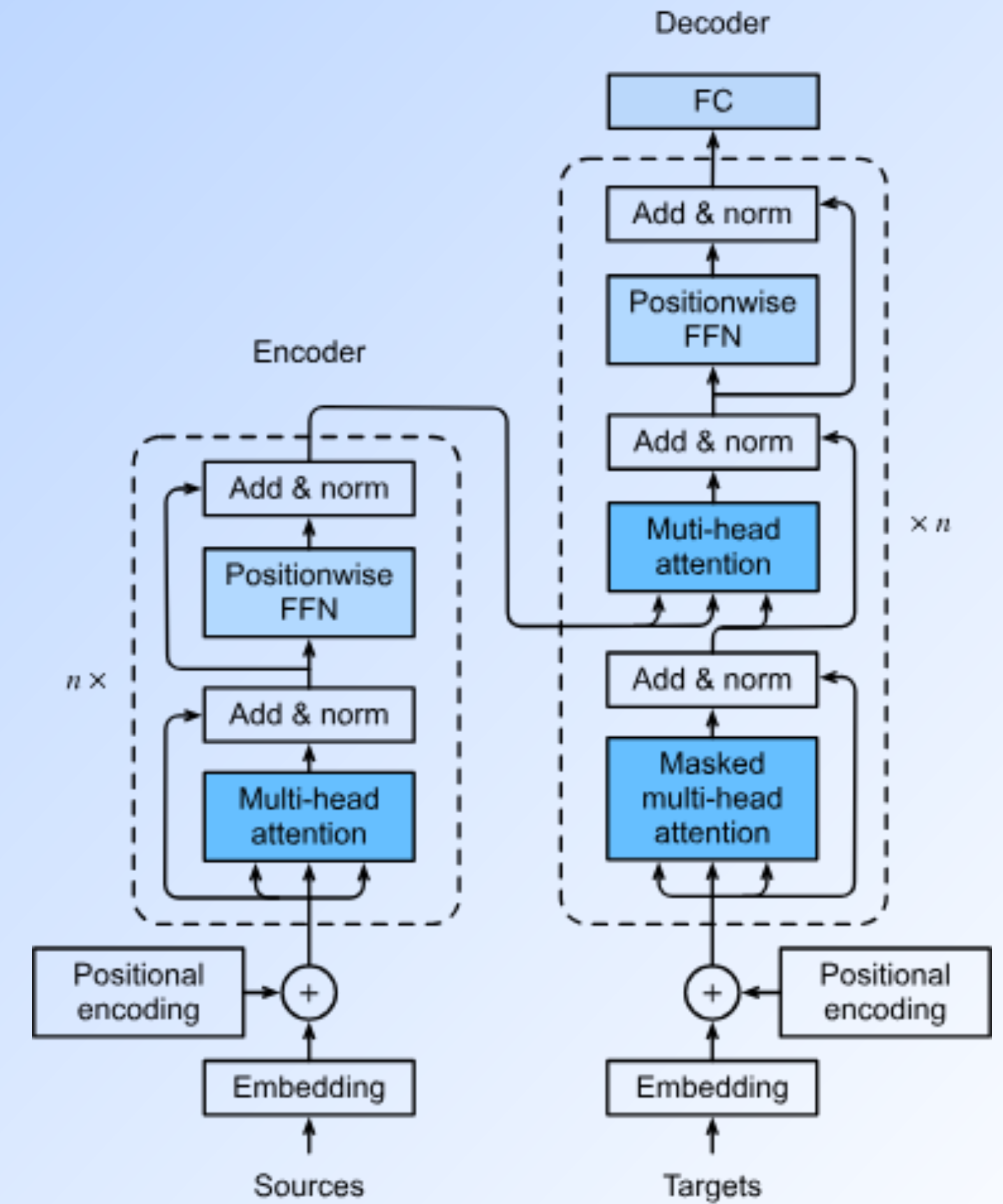


2015



Spatial Transformer Network

2



2017

Transformers for Sequence Modeling

- Transformers are a class of deep neural networks that have achieved state-of-the-art performance on sequence modeling tasks since 2017 (Vaswani et al., 2017).

- Key features of transformer models are:
 - Stacked encoder decoder architecture
 - Attention mechanism
 - Parallelization

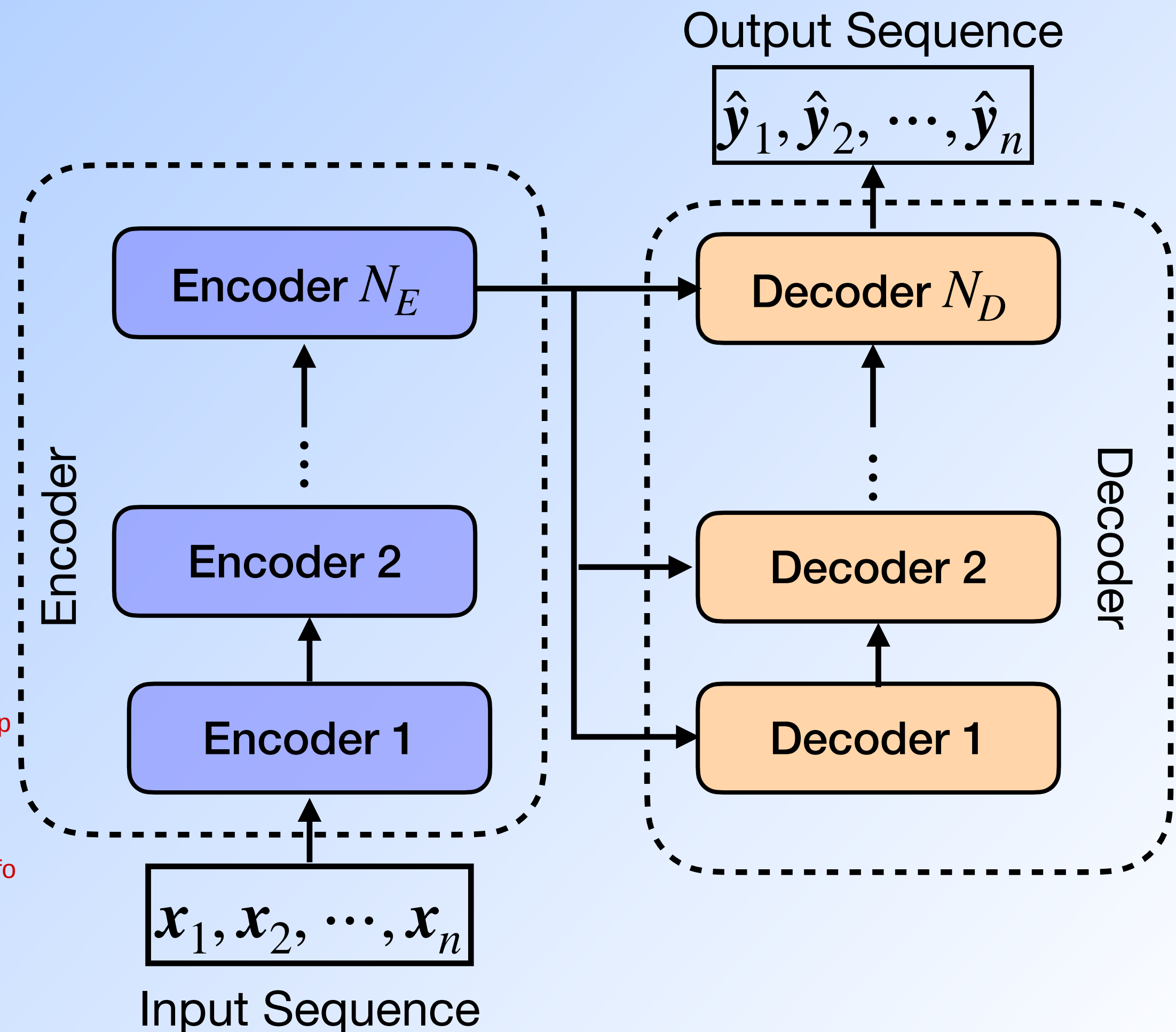
which allows it to weigh the relative importance of elements of the sequence in their context.

Transformers attention mechanism can compute how much a token x_1 is related to x_2 etc. Attention mechanism helps to compute the inter-relationship of each token with every other token in the given sequence.

The sequence $x_1, x_2, \dots, x_n$ goes through the encoder stack, encoder-stack consists of multiple encoder layers and sequence of input is processed by the stack of encoder layers to extract contextual information from the input sequence, then this contextual info is fed to the decoder, decoder is typically for sequence to sequence prediction tasks.

Decoder also consists of stack of decoder layers which processes this contextual information and something else.

Input and output sequence can be of different lengths, The number of layers in Encoder and Decoder can be different.



Input as Sequence of Tokens

- We will refer to the smallest unit of the input sequence that the transformer processes as a “token” (a term derived from NLP).
- An input sequence can be “tokenized” in multiple ways:
 - For text sequences: tokens can be chosen as words, sub-words or characters
 - For images: tokens can be chosen to be patches of arbitrary sizes.
 - For videos: tokens can be chosen to be frames or a group of frames of arbitrary length.
- For text data, tokens are mapped to their corresponding IDs using a predefined vocabulary to convert the data in numerical form.
- In addition to the tokens contained in the input sequence, special tokens are often used for classification, padding, masking, separator, unknown values, start and end of sequence, etc.

Tokens

40

Characters

204

Specifically, tokens are the segments of text that are fed into and generated by the machine learning model. These can be individual characters, whole words, parts of words, or even larger chunks of text.

when we are working with text in deep learning, models cannot directly process the raw text, so we need to convert raw text into some numerical representation using some pre-defined vocabulary which will assign unique id/ integer to each word or subwords.

Suppose we have a sentence: " I love deep learning"
We spilt the text into words as ["I" , "love" , "deep" , "learning"]

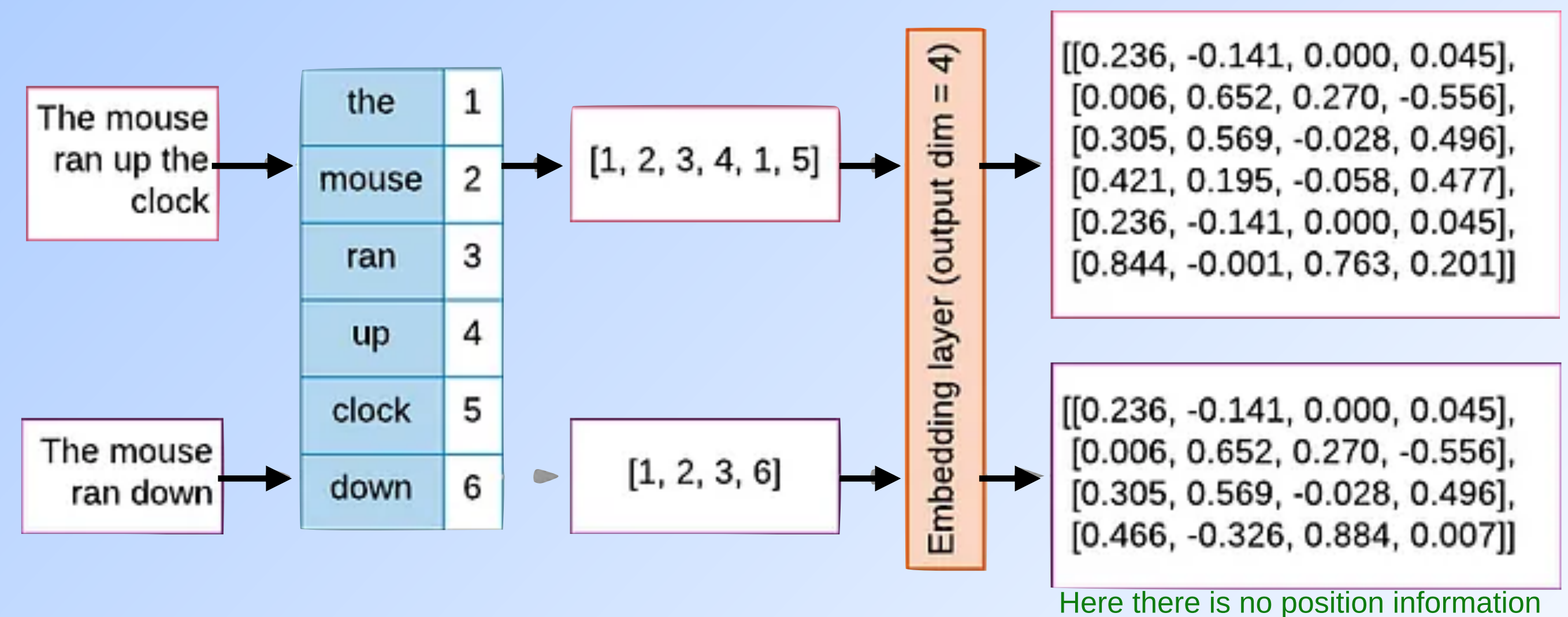
Then we create a vocabulary/ Dictionary that will assign unique id to each word as :
{ "I" : 1 , "love" : 2 , "deep" : 3 , "learning" : 4 }

Using the constructed vocabulary we will get the numerical representation of the sentence as :
[1, 2 ,3, 4]

Token Embeddings

- The token IDs are mapped to vectors by using a learned projection/embedding matrix.
- Token embeddings help to capture contextual meaning of tokens, and their relationships in the input sequence.
- The model learns to map tokens with similar contextual meaning to vectors that are close to one another in the high dimensional space.

Doubt :: matrix multiplication of H times one hot encoded vectors?
one hot encoded vector is not learnt or anything,
 $dL/dX = (dL/d y_hat) * (d y_hat / dX)$, apply chain rule and propagate the gradients back to the input matrix 'H'



We just have 2 sentences in the dataset, and we construct vocabulary with these 2 sentences. Vocabulary is just a unique set of words throughout the dataset; we give corresponding indexes to the word in the vocabulary, and we encode the sentences with the corresponding indexes of the words in the vocabulary, so we get the encoded sequences using indexes from the vocab. Then we have the embedding layer; this layer is learned by the model, which converts these indexes to corresponding vectors using the lookup table. There are 6 tokens above, and if the embedding dimension is 4, then each token is getting converted to a 4-dim vector space, so the size of the matrix is 6X4. This is based on the learnt matrix, or we can use word2vec models also for embedding but for transformer model these embeddings are learnt. For each word in the sentence, we need to learn a d-dimension vector.